DevSecOps-Oriented Vulnerability Detection

Integration of Security Analysis in the Software Development Lifecycle

Mehul Kapoor

Matricola: 555091

Supervisor(s): Prof. Francesco La Rosa

2026-05-21

Abstract

This internship project designed and implemented a lightweight DevSecOps prototype that integrates automated vulnerability detection into the software development lifecycle, following the shift-left security principle. Using GitHub Actions, two static analysis tools (CodeQL and Semgrep) were integrated into a continuous integration pipeline that scans a target repository on every push and pull request and reports findings, in the common SARIF format, directly within the development platform. The deliberately vulnerable Django application PyGoat served as the target repository. The work focused on the operational and usability comparison of the two tools rather than on building a vulnerability-detection benchmark: findings were categorized by type, severity, and redundancy; a representative subset was manually inspected to assess correctness and clarity; and the tools were compared on integration effort, execution overhead, feedback quality, and fit with CI/CD workflows. The evaluation found the two tools to be complementary. CodeQL offers deeper, more precise localization, while Semgrep offers broader coverage and more actionable remediation advice. The study also highlighted practical concerns such as alert fatigue and finding redundancy that any real-world adoption would need to address.

1 Introduction

Modern software is developed through collaborative, fast-moving workflows built around version-controlled repositories, pull requests, and automated pipelines. In this setting, treating security as a final checkpoint performed only before release is increasingly ineffective: by the time code reaches a dedicated security review, vulnerabilities may already be embedded throughout the codebase and its dependencies, and fixing them late is costly and disruptive.

DevSecOps responds to this problem by integrating security activities into the development process itself rather than adding them only at the end. A central idea within DevSecOps is **shift-left security**: moving security checks as early (as far "left") in the development timeline as possible, so that developers receive feedback while they are still writing and reviewing the code. A natural vehicle for this is the **CI/CD pipeline** (Continuous Integration / Continuous Delivery), the automated sequence of steps that runs whenever code changes, building and testing the software. By adding security analysis as a step in this pipeline, every code change can be automatically examined for vulnerabilities.

1.1 Goal and Scope

The goal of this internship project was to design and implement a lightweight, reproducible DevSecOps prototype that integrates security analysis into a CI/CD workflow, and to evaluate the integrated tools from a practical, operational perspective. The project specifically addresses how feasible it is to integrate multiple analysis tools into a single automated pipeline; how the tools compare in the volume, type, and quality of the feedback they produce; and what practical concerns arise when such a pipeline is considered for real-world use.

The scope was deliberately constrained to integration and comparison, rather than to the design of new detection techniques or a large-scale detection benchmark. The main contributions are:

- a working CI/CD pipeline, built on GitHub Actions, that automatically runs two analysis tools on every push and pull request.
- integration of CodeQL and Semgrep with unified, SARIF-based reporting inside the

GitHub Security interface.

- a categorized analysis of the findings produced on a real, intentionally vulnerable repository.
- a manual inspection of a representative subset of findings to assess correctness and clarity.
- a practical comparison of the two tools covering integration, overhead, feedback quality, and CI/CD fit, together with a critical discussion of the limitations of the approach.

2 Background and Related Work

Before describing the prototype, the ideas it rests on need to be established. The project does not invent a new detection method. Instead, it takes two existing, widely used analysis tools and studies how they behave once placed inside a real development pipeline. Understanding the results therefore requires three pieces of background. These are what static analysis actually does, the two different ways the chosen tools go about it, and the standards and test application used to make their outputs comparable.

2.1 Static Application Security Testing

The form of analysis used in this project is **Static Application Security Testing (SAST)**, the examination of source code *without executing it*, searching for patterns and data flows that indicate security weaknesses. Because SAST operates directly on source code, it fits naturally into a pipeline triggered by code changes and can provide early, automated feedback, which makes it well suited to the shift-left principle.

2.2 Two Paradigms of Static Analysis

Two broad families of static analysis are relevant to this project. The first is *semantic, dataflow-based analysis*, which builds a structured model of the program and reasons about how data moves from sources (such as user input) to sinks (such as a database query or a system command). **CodeQL** is a mature representative of this family and is integrated

natively into the GitHub platform. The second family is *rule-based pattern matching*, which scans code for syntactic patterns associated with known weaknesses. It is typically faster and easier to extend, but reasons less deeply about program behaviour. **Semgrep** is a widely used representative of this approach, distributed with community-maintained rule packs. Comparing one tool from each paradigm is central to this project.

2.3 Standards, Formats, and Test Subject

The vulnerability categories targeted throughout this work follow the widely adopted **OWASP Top 10** classification of common web-application risks and the **CWE** (Common Weakness Enumeration) taxonomy, both of which the tools reference in their output. For interoperability, this project relies on **SARIF** (Static Analysis Results Interchange Format), a standard JSON-based format for static-analysis results that GitHub uses to display findings from multiple tools in a single interface. The deliberately vulnerable application **PyGoat**, modelled on the well-known OWASP WebGoat project, served as the target repository. Such intentionally vulnerable applications are a standard means of exercising and demonstrating security tooling.

The project does not aim to outperform or replace these tools, but to study how they behave and compare when integrated into a realistic development workflow, an operational question that complements the detection-focused literature.

3 System Design and Implementation

This section describes the prototype that was built and how it works.

3.1 Assumptions and Requirements

The prototype was required to (i) trigger automatically on repository events (push and pull request), (ii) run more than one analysis tool, (iii) surface findings inside the development platform rather than in a separate dashboard, and (iv) remain lightweight and reproducible. All analysis ran on GitHub-hosted Ubuntu runners (Ubuntu 22.04, 2 vCPUs, 7 GB RAM) using Python 3 and the official tool distributions, so that timing and behaviour are reproducible from the committed workflow file. Success was understood as

a pipeline that runs reliably end to end and produces comparable, inspectable findings from both tools.

3.2 Target Repository

The target repository is **PyGoat**, an open-source Django web application deliberately constructed to contain vulnerabilities drawn from the OWASP Top 10, including SQL injection, server-side request forgery, and insecure deserialization. PyGoat was chosen because it provides a realistic application structure rather than isolated code snippets, while guaranteeing a rich set of vulnerabilities for the tools to detect. The repository was forked into a personal account. No application code was modified, so that the tools analyzed the application as-is.

3.3 Pipeline Architecture

The pipeline is defined as a single GitHub Actions workflow stored in the repository and triggered on every push and pull request to the main branch. It comprises three independent jobs that run in parallel on isolated virtual machines:

- a **sanity-check** job, created first, that verifies the workflow triggers correctly and the repository can be checked out, so that later failures can be attributed unambiguously to a specific tool rather than to the pipeline configuration.
- a **CodeQL** job, which initializes the CodeQL engine for Python, builds its analysis database, runs the `security-and-quality` query suite, and publishes results to the GitHub Security tab.
- a **Semgrep** job, which runs inside the official Semgrep container with four security-focused rule packs (`p/python`, `p/django`, `p/security-audit`, `p/owasp-top-ten`) and uploads its results to the same Security tab.

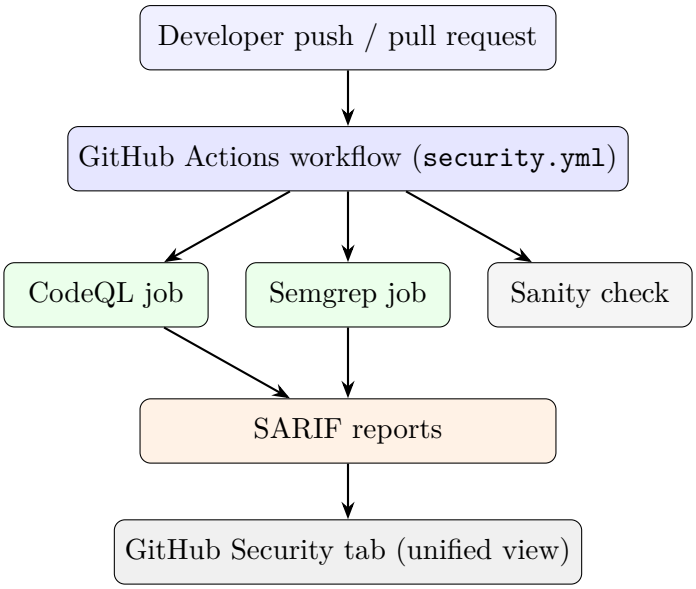

Figure 1: Pipeline architecture. A push or pull request triggers the GitHub Actions workflow, which runs three jobs in parallel; CodeQL and Semgrep generate SARIF, which is aggregated into the unified GitHub Security tab.

This pipeline runs entirely on GitHub's cloud infrastructure rather than on any local machine. When a push or pull request occurs, GitHub Actions reads the workflow file and provisions a fresh, isolated virtual machine for each job on demand. Each runner is a clean Ubuntu environment that exists only for the duration of its job, executes the defined steps, returns its SARIF output, and is then destroyed. The developer never manages these machines. The local machine only pushes the code that triggers the workflow. This ephemeral, cloud-hosted model is the standard lightweight option for GitHub Actions. A production deployment at a larger organization might instead use self-hosted runners, where the same workflow runs on machines the organization controls, typically for reasons of internal security, scale, or access to private resources. The pipeline logic is identical in either case. What changes is only where the runners physically execute.

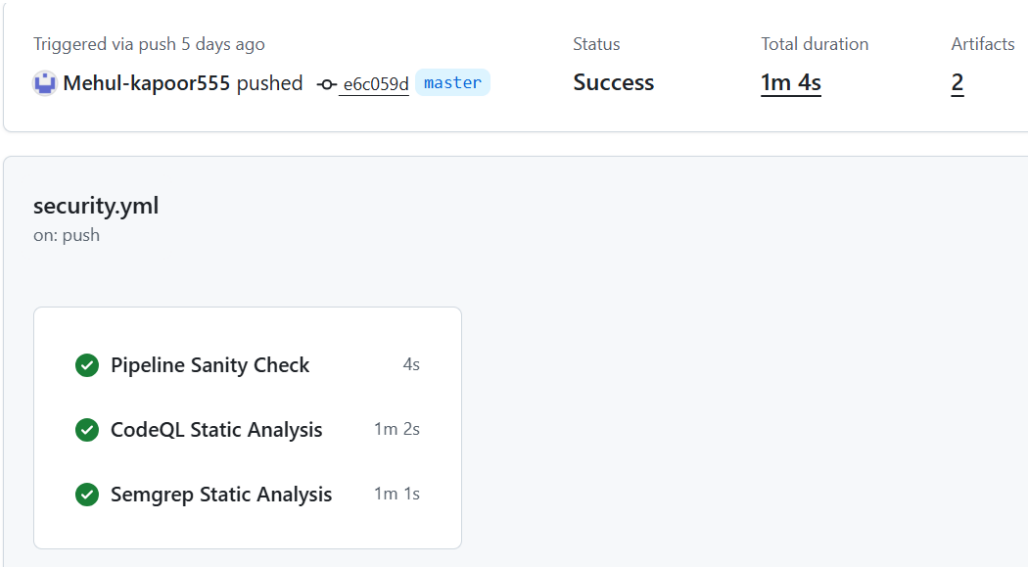

Figure 2: A successful pipeline run in the GitHub Actions interface, showing the three parallel jobs and their durations.

A condensed excerpt of the workflow illustrates the triggers and the parallel-job structure:

```yaml
on:
  push:
  pull_request:

jobs:
  sanity-check:   # verifies the workflow itself runs
    runs-on: ubuntu-latest
  codeql:         # dataflow analysis, publishes SARIF automatically
    runs-on: ubuntu-latest
  semgrep:        # pattern analysis in a container, uploads SARIF
    runs-on: ubuntu-latest
    container: { image: semgrep/semgrep }
```

Both tools report findings in the SARIF format. Using a common format is a deliberate design choice. It allows the output of two very different tools to be aggregated, filtered, and compared within a single interface, and it enables programmatic post-processing of the combined results. Running the tools as parallel jobs rather than sequentially keeps the total pipeline time close to that of the single slowest tool rather than the sum of both.

3.4 Implementation and Post-Processing

The pipeline was developed incrementally so that each component could be validated before further complexity was added. The trivial sanity-check workflow came first, then CodeQL, then Semgrep. After both tools were producing findings, their combined SARIF output was post-processed by a Python script. The script does four things. It rewrites every finding into one common format, it assigns each finding a vulnerability category (using the CWE identifier where available, and the rule name otherwise), it maps the two tools' different severity labels onto a single three-tier scale, and it separates genuine security findings from code-quality ones. It then computes cross-tool overlap, within-tool redundancy, and per-category and per-severity tallies.

To support reproducibility, the committed workflow file pins the analysis configuration. The specific PyGoat fork and commit, the CodeQL query suite (`security-and-quality`), the four Semgrep rule packs, and the runner operating system are all recorded there, so that the same scan can be re-run and is expected to produce the same findings.

For the redundancy analysis, two findings from the *same* tool were treated as a duplicate cluster when they fell in the same file within two lines of each other. This captures both the case of several rules firing on one line and the case of one rule firing across adjacent lines of a single statement. Every candidate cluster was then verified by hand against the source code.

4 Results

This section reports the factual results of running the pipeline on the PyGoat repository. Both tools analyzed the same codebase on the same commits. Findings were collected from the SARIF output and the GitHub Security interface, and timing was read from the GitHub Actions interface. Interpretation of these results is deferred to Section 5.

4.1 Finding Counts and Categorization

The two tools together produced 279 findings, 171 from CodeQL and 108 from Semgrep. These raw totals are not directly comparable, because CodeQL was run with the `security-and-quality` suite while Semgrep was run with security-focused rule packs. Separating the findings by intent clarifies the picture. Of CodeQL's 171 findings, only

39 are security findings, with the remaining 132 being code-quality or code-style findings. All 108 of Semgrep's findings are security findings (Table 1).

Table 1: Findings by intent. Raw totals are dominated by configuration differences; the security-only counts are the basis for comparison.

Kind	CodeQL	Semgrep
Security	39	108
Quality	49	0
Other (code style)	83	0
Total	**171**	**108**

The security findings span a wide range of categories (Table 2). Both tools detected the canonical injection vulnerabilities (SQL injection, command injection, code injection, SSRF, insecure deserialization, XXE) and weak cryptography. Beyond this common ground, each tool covered categories the other did not: Semgrep additionally reported cross-site scripting in templates, Dockerfile misconfigurations, hardcoded credentials, and a large number of Django CSRF-configuration findings, while CodeQL additionally reported log injection, clear-text logging of sensitive data, and path traversal.

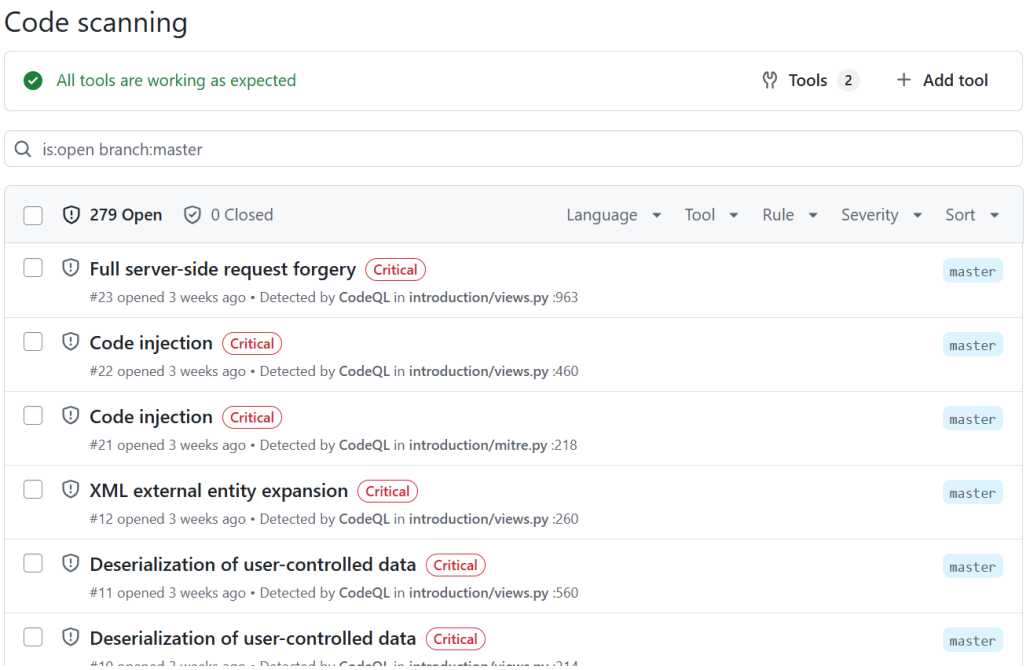

Figure 3: The unified GitHub Security tab showing findings from both tools, filterable by tool, rule, and severity.

Table 2: Security findings by category.

Category	CodeQL	Semgrep
CSRF (configuration)	0	44
Cookie Security	6	12
Cross-Site Scripting	0	10
Insecure Deserialization	3	8
Framework Misconfiguration	3	8
Weak Cryptography	4	6
Command Injection	5	5
Code Injection	2	4
Dangerous File Write	0	4
XXE	1	3
SQL Injection	2	2
SSRF	1	1
Hardcoded Credentials	0	1
Sensitive Data Exposure	6	0
Log Injection	4	0
Path Traversal	1	0
XML Security	1	0

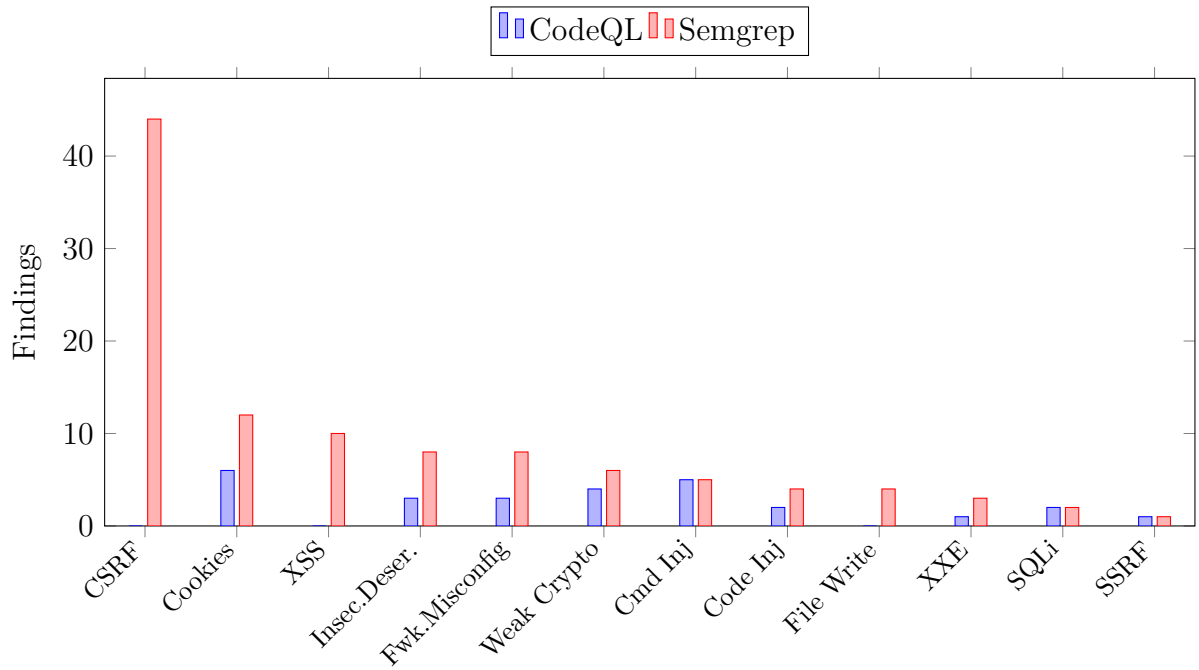

Figure 4: Security findings by category, CodeQL vs. Semgrep. The chart makes the complementary coverage visible: the tools converge on the canonical injection categories (right) but diverge sharply on framework- and configuration-specific categories (left).

Restricting to security findings, the two tools agree closely at the most serious end — 24 high-severity findings for CodeQL versus 19 for Semgrep — with their differences

concentrated in the medium tier.

Table 3: Severity normalization. The two tools' native vocabularies are mapped onto a common three-tier scale.

Normalized	CodeQL native	Semgrep native
High	Critical, High	Error
Medium	Medium, Warning	Warning
Low	Low, Note	Info

Table 4: Severity distribution of security findings (normalized to a common three-tier scale).

Severity	CodeQL	Semgrep
High	24	19
Medium	15	89
Low	0	0

4.2 Descriptive Statistics and Metric Applicability

Several descriptive statistics summarize the comparison. On security findings, Semgrep produced 2.8 times as many as CodeQL (108 against 39). Within-tool redundancy affected 36 percent of CodeQL's security findings and 25 percent of Semgrep's. Cross-tool overlap accounted for 12 shared detections. Of these 12, the two tools assigned the same normalized severity in 6 cases and disagreed in the other 6. In the manually inspected subset of 17 findings, 16 were confirmed true positives, giving an observed true-positive rate of about 94 percent on that subset.

It is important to be clear about which statistics can and cannot be computed here. Standard detection metrics such as precision, recall, and F1 require a complete ground-truth labelling of the target, that is, an authoritative list of every vulnerability that is present and every location that is not. PyGoat documents some of its planted vulnerabilities but does not provide such a complete, verified labelling of the entire codebase. Recall cannot be computed without knowing the full set of vulnerabilities actually present, and tool-wide precision cannot be computed without manually verifying every one of the 279 findings rather than a representative subset.

For this reason, this study reports the true-positive rate on the verified subset and descriptive statistics on the full finding set, and it does not report precision, recall, or F1

for the tools as a whole. Computing those metrics would require introducing unverifiable assumptions about findings that were never manually checked.

4.3 Redundancy: Overlap and Duplication

Cross-tool overlap, the same issue independently flagged by both tools, occurred in twelve cases (by category and file). *Within-tool duplication*, the same tool reporting the same underlying issue more than once, was also significant. Roughly one in three of CodeQL's security findings and one in four of Semgrep's belong to a duplicate cluster (Table 5).

Table 5: Within-tool duplication of security findings (manually verified against source).

Metric	CodeQL	Semgrep
Security findings	39	108
Duplicate clusters	7	12
Findings inside a cluster	14	27
Share of output duplicated	36%	25%
Dominant cause	Multi-property queries	Rule-pack stacking

In Table 5, "Findings inside a cluster" counts the individual alerts that belong to a duplicate group rather than standing alone. For CodeQL, 14 of its 39 security alerts collapse into 7 distinct problems, and for Semgrep, 27 of its 108 collapse into 12. The remaining alerts in each tool are unique. This is the sense in which the raw alert count overstates the number of distinct issues a developer must address.

4.4 Execution Time

Two representative pipeline runs were measured (Table 6). CodeQL was stable across the two runs, at 1 minute 9 seconds and 1 minute 2 seconds, while Semgrep varied more, at 40 seconds and 1 minute 1 second. In both runs the total pipeline duration was close to that of the slower single job rather than the sum of the two. The variation in Semgrep's time reflects container startup overhead rather than the scan itself, which is fast on a codebase of this size.

Table 6: Measured execution times across two pipeline runs.

Run	CodeQL	Semgrep	Total pipeline
1	1m 9s	40s	1m 21s
2	1m 2s	1m 1s	1m 4s

5 Analysis and Discussion

This section interprets the results. It covers what the categorization and redundancy figures mean, what a manual inspection of selected findings revealed, and how the two tools compare across the operational dimensions that matter for a DevSecOps workflow.

5.1 Interpreting the Categorization

The most important interpretive point is that the apparent lead of CodeQL in raw volume (171 against 108) is an artifact of configuration. Once code-quality and code-style findings are separated out, CodeQL contributes only 39 security findings against Semgrep's 108, so on security findings alone Semgrep is the broader-coverage tool. Coverage is also not symmetric. Each tool detects whole categories the other misses entirely (Table 7).

Table 7: Categories detected by only one tool. Coverage is complementary rather than overlapping.

Only CodeQL detected	Only Semgrep detected	Both detected
Sensitive Data Exposure	Cross-Site Scripting	SQL Injection
Log Injection	CSRF (configuration)	Command Injection
Path Traversal	Dockerfile Misconfig.	Code Injection
XML Security	Hardcoded Credentials	SSRF
	Dangerous File Write	Insecure Deserialization
		XXE, Weak Crypto, Cookies

This pattern of difference is explained by how each tool works. The categories unique to Semgrep are largely framework-specific and configuration-specific, such as cross-site scripting through Django's `|safe` filter, missing CSRF tokens, and a Dockerfile running as root. These are surface patterns that a rule written specifically for Django or for Dockerfiles can match directly, and Semgrep ships exactly such framework-specific rule packs. The categories unique to CodeQL, such as log injection, clear-text logging of sensitive data, and path traversal, are the cases where a finding depends on following

a value through the code to where it is used. That is what CodeQL's dataflow-based analysis is built to do, and what a single-line pattern match tends to miss.

The agreement on the remaining categories is equally explainable. The vulnerabilities both tools caught, namely SQL injection, command injection, code injection, SSRF, insecure deserialization, and XXE, are the canonical weaknesses for which both a mature dataflow query and a community pattern rule reliably exist. The tools agree on the textbook cases that every analysis approach covers, and diverge where their underlying technique gives them an advantage. The complementarity is therefore not coincidental. It is a direct consequence of pairing a dataflow tool with a pattern-matching tool.

5.2 Interpreting the Redundancy

The twelve cross-tool overlaps, agreed upon by two tools using different methods, are the highest-confidence findings in the dataset. The within-tool duplication is more revealing. Rather than only counting it, it helps to see what was actually duplicated. Table 8 lists representative cases from each tool.

Table 8: Representative duplicate findings within each tool.

Tool	Location	Alerts	What was duplicated
Semgrep	`views.py:430-432`	3	One `subprocess` call flagged by three command-injection rules from three packs
Semgrep	`views.py:17-19`	3	Three consecutive XML imports each flagged for the same unsafe-parser issue
Semgrep	`views.py:214`	2	One `pickle` call flagged by both a Django and a general-Python rule
CodeQL	`views.py:291, 305, 319`	6	Three cookies, each flagged once for a missing flag and once for client exposure
CodeQL	`views.py:260`	2	One XML parser flagged once as XXE and once as an XML-bomb risk

These examples show that the two tools repeat themselves for different reasons (Table 9). Semgrep's duplication comes from *stacked rule packs*. Because the four configured packs were authored independently and overlap in coverage, the same construct is matched several times. The single `subprocess` call above triggered three separate rules. CodeQL's duplication comes from its *property-oriented queries*. A single misconfigured cookie is flagged once for a missing `Secure` flag and again for being exposed to client-side scripts, because these are checked by separate queries.

Table 9: Why each tool duplicates — and therefore how each could be reduced.

Tool	Cause of duplication	How it could be reduced
Semgrep	Stacked rule packs overlap in coverage	At configuration time, by trimming overlapping packs
CodeQL	Separate queries flag separate properties of the same code	Not reducible by configuration; handled during triage

The distinction is practical. Semgrep's redundancy is a side effect of a user choice, the stacking of four packs for coverage, and can be tuned away, though doing so reduces coverage. CodeQL's redundancy is built into how it reasons and cannot be removed by configuration. From the developer's standpoint the effect is the same. As Table 5 reported, roughly a third of CodeQL's security findings and a quarter of Semgrep's are duplicates, so the alert count overstates the number of distinct problems in both cases.

5.3 Manual Inspection of Representative Findings

To check whether the tools were actually correct and how clearly they communicated, a subset of seventeen findings was inspected directly against the source code. The subset was chosen purposively rather than at random, to cover each axis of interest. It included roughly five cross-tool agreements (the highest-confidence cases), several findings unique to each tool, two of the highest-frequency rules suspected of over-firing, and a few ambiguous edge cases where exploitability was unclear. This spread was intended to test the tools where their behaviour was most likely to differ. The verdicts are summarized in Table 10.

The tool-unique findings were instructive. CodeQL's unique findings, such as log injection and the clear-text logging of a password, were genuine issues Semgrep did

Table 10: Outcome of manually inspecting seventeen representative findings.

Observation	Count	Notes
Confirmed true positives	15	As expected for a vulnerable app
Confirmed false positive	1	CSRF flagged on already-protected form
Technically correct but low-value	1	`csrf-exempt` on a read-only endpoint
Cross-tool agreements, all genuine	5	Highest-confidence findings
Duplicates also caught by hand	several	Confirms redundancy is developer-visible

not cover. Semgrep's unique findings included both genuine issues outside CodeQL's configured scope, such as a Dockerfile running as root and a hardcoded JWT signing secret, and the one clear false positive, a CSRF warning on a form that already contained the required token, which Semgrep raised because it matched the form pattern without accounting for the nearby token. The single false positive and the one low-value finding together illustrate why raw output cannot be trusted blindly.

5.4 Comparative Analysis

The manual inspection of individual findings made a direct comparison possible. Having read each tool's actual output against the source code, the two could be weighed against each other across the five operational dimensions that matter most for a DevSecOps pipeline. Table 11 summarizes this comparison, and the paragraphs that follow expand on each dimension.

Actionable feedback. The two tools provide different kinds of help, confirmed directly during inspection. Semgrep's messages were consistently the more actionable in prose. They name the risk and recommend a concrete remediation. For the raw SQL query it advised parameterized queries and the Django ORM, and for the `pickle` call it advised serializing as JSON instead. CodeQL's messages, by contrast, were brief and templated, reporting a variant of "this operation depends on a user-provided value" without suggesting a fix. CodeQL findings carry something Semgrep findings did not, a clickable "Show paths" control in the GitHub Security interface that, when expanded, displays the full dataflow from the untrusted source to the dangerous sink in detail. Semgrep findings offered no equivalent. They point at a single line with no traceable path. The two tools therefore assist at different stages. Semgrep's text is more useful for deciding how to fix an issue, while CodeQL's path view is more useful for understanding

Table 11: Head-to-head comparison across operational dimensions.

Dimension	Semgrep	CodeQL
Feedback	Names the risk and recommends a concrete fix, but no path view	Brief message, but a clickable "Show paths" button reveals the full source-to-sink dataflow
Integration	More setup: runs in a container, needs rule-pack selection and an explicit upload step	Easier: native GitHub support, no container, results published automatically
Noise source	Over-firing config rules and rule-pack overlap	Code-quality rules from the `security-and-quality` suite
Clarity	Long, plain-language messages; cryptic rule IDs	Short messages; human-readable rule names
CI/CD fit	Fast; suits per-pull-request checks	Stable ∼65s; grows with code size; suits scheduled scans

how the vulnerable data reaches the sink. Neither alone gives a developer both.

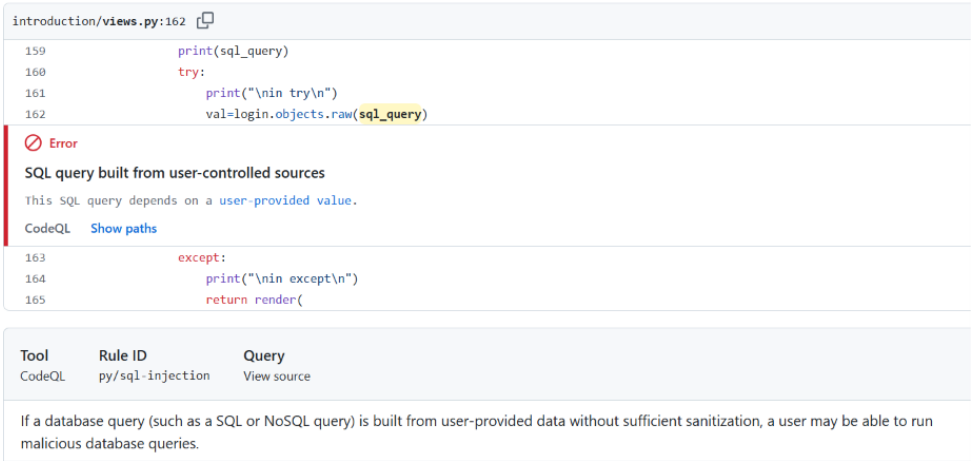

Figure 5: A CodeQL finding with its "Show paths" dataflow trace expanded — a feature Semgrep findings did not provide.

Integration effort. CodeQL was the easier tool to integrate, and the difference was concrete. CodeQL is supported natively by GitHub. It was enabled through its official action, required no container, and published its results to the Security tab automatically. Semgrep required noticeably more setup. It runs inside its own Docker container

(`semgrep/semgrep`), its four rule packs had to be chosen and configured explicitly, and its results needed an explicit upload step, including a flag to stop a non-zero exit code (returned whenever findings exist) from failing the whole pipeline. The trade-off is that this extra configuration is also what gave Semgrep its broader coverage. The effort buys something.

Noise. Both tools are noisy, but the source of the noise is different and measurable. CodeQL's noise is dominated by code-quality rules pulled in by the `security-and-quality` suite, namely 35 "unsafe exception handling" findings and 14 "resource management" findings, 49 in total, none of which are security issues and all of which can be removed by filtering on severity or by switching to a security-only query suite. Semgrep's noise is different in kind. It comes from a small number of security rules firing very often, led by `csrf-exempt` (25 firings) and `django-no-csrf-token` (19 firings), which together account for 44 of its 108 security findings. This noise cannot be filtered by severity, because the findings are genuine security warnings. It can only be reduced by tuning or disabling specific rules. CodeQL's noise is therefore the easier of the two to manage.

Clarity. The inspected messages showed a clear difference. Semgrep's findings carry long, plain-language explanations of the risk, but are identified by cryptic hierarchical rule IDs such as `python.django.security.injection.ssrf.ssrf-injection-requests`, which mean little to a developer at a glance. CodeQL's messages are shorter, but its rule names are human-readable, for example "SQL query built from user-controlled sources", so a developer can grasp the category immediately even if the message gives no remediation. Neither is unambiguously clearer. Semgrep is clearer on what to do, CodeQL is clearer on what kind of problem it is. For a developer scanning a long list of alerts, CodeQL's readable names aid triage. For a developer who has already opened one finding, Semgrep's message aids the fix.

CI/CD fit. Both were fast enough on PyGoat. CodeQL's time was stable at about 65 seconds and is dominated by a database-build step that grows with code size. Semgrep's pattern matching is faster and scales more gently. At larger scale this suggests a division of labor, with Semgrep as a fast per-pull-request check and CodeQL as a deeper scheduled scan. Because the jobs run in parallel, adding the second scanner cost almost no extra wall-clock time.

5.5 Workflow and Usability

Considered overall, the prototype delivers real shift-left value. Security feedback appears automatically, early, and within the developer's own platform through the unified SARIF view, which displayed both tools and allowed filtering by tool, rule, and severity. The integration was not entirely seamless. Semgrep required its own container and an explicit upload step, and the tools' severity vocabularies did not align in the unified view. Even so, SARIF made multi-tool integration practical. The principal usability concerns are the volume and redundancy of findings, addressed in the next section.

6 Limitations and Critical Reflection

The results above must be read against several limitations, some intrinsic to the technique and some specific to this study.

Limitations of static analysis. Static analysis reasons about code without executing it, and so cannot confirm that a flagged issue is genuinely exploitable. It cannot know whether a vulnerable path is reachable at runtime or whether validation elsewhere neutralizes the input. The manual inspection illustrated this directly. The inspected `csrf-exempt` finding was technically correct yet of limited practical risk, because the endpoint performed no sensitive action. The single confirmed false positive, a CSRF warning on an already-protected form, further shows that pattern-based detection inevitably trades precision for coverage. The two tools sit at different points on this spectrum, but neither escapes the underlying constraint.

Absence of ground truth and its effect on metrics. A direct consequence of using PyGoat is that the study cannot report standard detection metrics for the tools as a whole. Precision, recall, and F1 all require a complete and authoritative labelling of the target, meaning a full list of every vulnerability that is present and every location that is not. PyGoat documents many of its planted vulnerabilities but does not provide such a complete labelling. Recall is therefore not computable, because the full set of vulnerabilities actually present is unknown, and tool-wide precision is not computable without manually verifying all 279 findings rather than a representative subset. This is why the results report a true-positive rate only on the manually verified subset, together with descriptive statistics on the full finding set. Reporting precision, recall, or F1 for

the tools as a whole would require unverifiable assumptions about findings that were never checked, which would undermine rather than strengthen the analysis. A controlled dataset with known ground truth, rather than an intentionally vulnerable application, would be required to compute those metrics meaningfully.

Limitations of an intentionally vulnerable repository. PyGoat was an appropriate target, but its design constrains what the results mean. Its vulnerability density is unrealistically high, with 279 findings on a single small application, whereas professionally written code yields far sparser results. Its vulnerabilities are also canonical and textbook in nature, exactly the patterns SAST detects best, so the high true-positive rate observed here would not transfer to the subtler logic and validation flaws of production code. Some findings, such as the intentional Flask debug-mode configuration, are correct detections of deliberately dangerous code. Conclusions about how the tools behave and compare are therefore valid, while conclusions about production hit-rates are not.

Alert fatigue. The volume and redundancy of findings pose a real risk that developers begin to ignore the output. With 279 findings on one application, and with a notable fraction of each tool's output being duplicated, the number of alerts substantially overstates the number of distinct problems to fix. Findings that are technically correct but low in value, such as broad `csrf-exempt` firings, dilute attention further. Realistic adoption would require severity gating, de-duplication, and baseline diffing, none of which the raw tooling supplies automatically.

Depth versus CI/CD usability. Finally, there is a real tension between analytical depth and pipeline usability. CodeQL's deeper analysis is more precise, but its database-build step grows with code size, pushing it toward scheduled scans on large codebases. Semgrep's shallower matching is faster and better suited to per-pull-request feedback, at the cost of more false positives. Greater depth did not translate into more actionable feedback. CodeQL localized issues more precisely, yet Semgrep gave better remediation advice. Depth and feedback usability are therefore partly independent, and no single point on the trade-off is universally optimal.

7 Conclusion

This internship project produced a working, lightweight DevSecOps prototype that integrates two static-analysis tools into a GitHub Actions pipeline, scans a real and intentionally vulnerable repository on every code change, and surfaces unified, SARIF-based feedback within the development platform. The evaluation, centred on operational and usability concerns, found CodeQL and Semgrep to be complementary rather than competing. CodeQL offers deeper, more precise diagnosis and richer dataflow context, while Semgrep offers broader coverage and more actionable remediation advice. Each detected real vulnerabilities the other missed, which argues for using them together rather than choosing between them. The custom SARIF post-processing framework further improved the reproducibility and analytical depth of the evaluation by enabling automated normalization, categorization, overlap analysis, and report generation across multiple SAST tools.

The evaluation also made clear that the value of such a pipeline is bounded by the limitations discussed in Section 6, namely the inherent imprecision of static analysis, the absence of ground truth on an intentionally vulnerable target, the risk of alert fatigue, and the tension between analysis depth and CI/CD usability. These point to concrete next steps for real-world adoption. They include combining complementary tools rather than relying on one, applying severity gating, de-duplication, and baseline diffing to control noise, and providing a human triage process, since neither tool's raw output is developer-ready on its own. The DiverseVul vulnerability dataset, considered earlier in the project, remains available as a complementary resource for a more controlled, ground-truth-based evaluation of detection accuracy in future work. Overall, the project shows that integrating SAST into a CI/CD workflow is both feasible and valuable, and that responsible deployment depends on understanding and managing the boundaries of the approach.